Software Requirements Specification

for

The Turning

version <1.0>

Prepared by

Group Number: 11

Akila Fernando	30169955	Team Lead
Abia Jahangir	30208171	Team Member
Ayre Ibrahim	30207941	Team Member
Parvathi Madhu	30174845	Team Member
Sabrina Li	30173044	Team Member

Course: ENSF 400

Instructor: Dr. Abdel Latif

Date: February 9th 2026

***Abstract*—This project proposes a text-based horror game that integrates AI generated and human written dialogue as a core gameplay mechanic. Players must survive by identifying whether ingame characters dialogue originates from a human author or an LLM, using narrative cues and contextual inconsistencies rather than visual indicators. The proposed system operates as a standalone, Unity run application and serves as social commentary in AI assisted storytelling, examining how effectively LLMs can mimic human narrative voice within an emotionally engaging environment.**

Keywords—LLM, Large Language Models, AI-Assisted storytelling, Text-Based game, Interactive

I. Introduction

Large language models have advanced rapidly in recent years, demonstrating an unprecedented ability to generate coherent, context aware, and almost human like text. As their adoption increases, a critical question emerges, at what point can we stop telling the difference between human and AI generated texts, particularly in emotionally charged or narrative driven contexts? This project explores human and AI text indistinguishability by embedding LLM generated dialogue alongside human written content in an interactive narrative setting, where users must rely solely on linguistic and contextual cues to successfully finish the game. The proposed system uses Unity as its base engine for the experience with an emphasis on text only content as its form of narrative with an addition of images to better immerse the player. Horror like narratives naturally emphasize ambiguity, tension, and psychological discomfort, making them suited for observing how users interpret subtle differences in tone, pacing, and coherence. As such, this project positions interactive storytelling not merely as entertainment, but as a practical framework for examining the strengths and limitations of large language models in mimicking human-written dialogue.

II. Detailed description

A. Product Perspective

The system is designed as a standalone system based on the Unity engine. It operates independently while leveraging external large language model APIs for dynamic dialogue generation. All game logic, narrative flow, and state management are handled within the application itself. The system emphasizes modularity to allow future expansion of story modules or AI providers.

B. User Base, Stakeholders and Characteristics

The primary player base are those who enjoy text based interactive sim games with slight use of horror elements. Users would need to have basic computer literacy and the ability to read and interpret narrative dialogue. No administrative or privileged user roles are required, as the system assumes a single local user per installation. All users interact with the system through the same interface and feature set.

C. Environment

The project will be developed using the Unity game engine and deployed as a standalone product that can be used by any user with a system running Windows or macOS. Unity's runtime environment will manage user input, audio playback, and narrative flow. An active internet connection is required to enable communication with the external LLM API for dynamic dialogue generation. The system relies on local file storage for saving game progress and configuration data.

D. Design and Implementation Constraints

The system is constrained by the performance and availability of external LLM services, including API response times and usage limits. Technology choices such as Unity, JavaScript, HTML, and CSS impose limitations on rendering, storage, and platform support. As the game is primarily text-based, visual elements are intentionally minimal. Additionally, the system must comply with API usage policies and licensing requirements.

E. Assumptions and Dependencies

The system assumes that users have a stable internet connection and a compatible device capable of running modern applications. It also assumes that the selected LLM service remains available and responsive during gameplay. The application depends on external APIs for AI-generated dialogue and on local file storage for saving game progress.

III. System Features

These would be tentative features the user can hope to see in the system. The list will be ordered in terms of priority per feature set. The features would include but not be limited to:

A. Game Architecture (GA)

1. Modular narrative system that separates story logic and dialogue content.
2. Local save file management to store player progress and preference. Done using structured formats
3. A Fullscreen experience complete with stylised texts and sounds
4. Integration with a large language model via API calls for dynamic dialogue generation or saved dialogue states to save memory use
5. Directed Acyclic Graph used for defined dialogue paths per NPC
6. Program will have an "out-of-box", pre generated AI responses to save on first launch resources

B. Interface (GI)

1. System will feature a starting menu. Players can choose to continue or start a new game, adjust settings, or close the application.
2. A difficulty setting will be available for the user to adjust how easy/hard it is to detect AI text. Can be changed during gameplay
3. User can exit current session during gameplay anytime using the pause menu

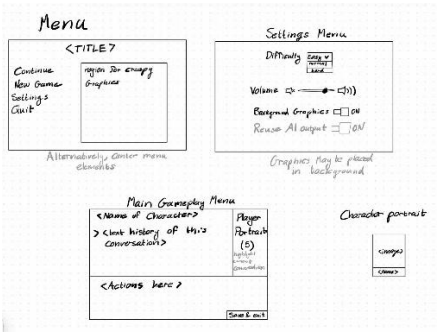

Figure 1: Conceptual Design for the user interactive menus

C. Gamplay (GP)

1. User will interact with the application primarily using the mouse pointer and preset shortcuts
2. User will choose dialogue options that are pre-determined while NPC dialogue can be both pre-made or LLM made
3. User can aim to get the role (Human or AI) of the NPCs determined by the end of the game. An end of game session screen will be displayed to showcase player statistics.
4. User can choose to use "out-of-box" pre-generated AI-responses or generate new responses for a new game session

IV. Requirements

A. External Interface

- The system will be mostly (if not entirely) text-based with audio cues when actions are taken. The wireframe images will be included in this document as concept art.
- Unity with C# will be used to develop the game, with exact libraries within Unity TBD. As for LLMs, ChatGPT would be an ideal choice for its simplicity and price. No databases or logins will be used, hence, the game's save state will be stored entirely locally.
- The program will have to communicate with the LLM through API calls, since it local hosting is not possible at current scope. Otherwise, there is minimal external communication from the game. Since any relevant save state will be stored locally, the game will also have to read and write files on local drives.

B. Non-Functional

- System will not include any security of any kind as program will be standalone and with prompt user to input any personal information
- Users with standard computer and game literacy can use the program
- Program will require a serviceable Wi-Fi connection during gameplay for the LLM
- Users can hope to complete a session within 5-15 minutes.